

Relatório | Grafos- INE5413

Alunos:

- Davi Ludvig - 23100473
- Pedro Henrique Gimenez - 23102766
- Victória Rodrigues - 23100460

Execício 1

Para a resolução do primeiro exercício, as funções `cfc`, `visita` e `exibir` foram construídas.

- `cfc(grafo: GrafoDirigido)`: Essa função é responsável por coordenar o processo de identificação das **Componentes Fortemente Conexas (CFCs)** do grafo. A principal funcionalidade dela é realizar a chamada das outras funções. Primeiramente, executa uma ordenação topológica (função `ord_topologica` de A2_2) do grafo, o que faz obter a ordem decrescente dos tempos de término dos vértices. Em seguida, inverte as arestas do grafo e, para cada vértice ainda não acessado, cria uma nova CFC e chama a função `visita`.
- `visita(grafo : GrafoDirigido, v : Vertice, componente : list)`: Marca o vértice `v` como acessado, armazena `v` na componente `e`, em seguida, explora todos os seus vizinhos. Para cada vizinho que ainda não foi acessado, a função é chamada **recursivamente**.
- `exibir(grafo: GrafoDirigido) -> None`: Esta função é **responsável por exibir as CFCs identificadas**. Ela organiza os vértices em conjuntos, onde todos os vértices de uma mesma componente conexa estão agrupados. Para cada vértice, verifica se ele é uma raiz de componente (não possui antecessor) e, em seguida, imprime os vértices pertencentes a cada componente em uma linha.

Exercício 2

Para a resolução do problema de ordenação topológica, as funções `ord_topologica`, `visita` e `exibir` foram construídas.

- `ord_topologica(grafo: GrafoDirigido) -> list`: Esta função é responsável por realizar a **ordenação topológica de um grafo dirigido**. Começa inicializando uma lista vazia `o`, que armazenará os vértices na ordem topológica. Para cada vértice `u` no grafo, se ele ainda não foi acessado, a função `visita` é chamada para realizar a busca em profundidade. Após visitar todos os vértices, a lista `o` é retornada com os vértices na ordem desejada.
- `visita(grafo: GrafoDirigido, v: Vertice, o: list) -> None`: Esta função realiza a **visita recursiva** a um vértice `v` durante a busca em profundidade. Primeiro, o vértice `v` é marcado como acessado. Em seguida, a função itera sobre todos os vizinhos do vértice (representados por $N^+(v)$). Para cada vizinho `u` que ainda não foi acessado, a função `visita` é chamada recursivamente. Após visitar todos os vizinhos, o vértice `v` é inserido no início da lista `o`, garantindo que os vértices sejam adicionados na ordem **correta** para a ordenação topológica.
- `exibir(ord: list) -> None`: Esta função é responsável por **exibir a ordem topológica** resultante de forma legível. Ela itera sobre a lista de vértices ordenados, imprimindo os rótulos de

cada vértice na ordem topológica. Os rótulos são separados por " → ", e o último rótulo é seguido por um ponto final, formando uma apresentação clara da sequência de atividades ou passos representados pelos vértices do grafo.

Exercício 3

Para a resolução deste problema, as classes e funções `cdElemento`, `cdLigar`, `cdEncontra`, `cdUniao`, `cdMesmoConjunto`, `ordenar`, `kruskal`, e `exibir` foram construídas.

- **Classe `cdElemento`:** Esta classe representa um elemento em uma estrutura de conjuntos disjuntos, usada para realizar a união e a busca de conjuntos durante o algoritmo de Kruskal. Cada elemento tem um ponteiro para seu pai (`pai`) e uma classificação (`rank`), que auxilia na operação de união, minimizando a altura das árvores disjuntas.
- **`cdLigar(x: cdElemento, y: cdElemento) -> None`:** Realiza a união de dois conjuntos representados por seus elementos `x` e `y`. Se a classificação (`rank`) de `x` for maior, `y` torna-se descendente de `x`. Caso contrário, `x` se torna descendente de `y`, e, se ambos tinham o mesmo `rank`, o `rank` de `y` é incrementado.
- **`cdEncontra(x: cdElemento) -> cdElemento`:** Realiza a busca com compressão de caminho, retornando o representante do conjunto ao qual o elemento `x` pertence. Durante a busca, atualiza os ponteiros de todos os elementos no caminho para apontarem diretamente para o representante, reduzindo a complexidade de futuras operações.
- **`cdUniao(x: cdElemento, y: cdElemento) -> None`:** Realiza a união dos conjuntos que contêm os elementos `x` e `y` por meio da função `cdLigar`, unindo os dois conjuntos disjuntos.
- **`cdMesmoConjunto(x: cdElemento, y: cdElemento) -> bool`:** Verifica se dois elementos `x` e `y` pertencem ao mesmo conjunto, utilizando `cdEncontra` para comparar os representantes dos conjuntos.
- **`ordenar(arestas: dict) -> list`:** Ordena as arestas do grafo pelo peso, retornando uma lista de arestas organizadas de acordo com seus pesos em ordem crescente. A função utiliza a função `sorted` para ordenar o dicionário de arestas.
- **`ruskal(grafo: GrafoDirigido) -> list`:** Esta é a função principal que implementa o algoritmo de Kruskal para encontrar a árvore geradora mínima de um grafo ponderado e não-direcionado. Ela realiza os seguintes passos:

1. **`**Inicialização da Lista de Arestas**`**
 - Cria-se uma lista ``a`` que armazenará as arestas da árvore geradora mínima.
2. **`**Criação dos Conjuntos Disjuntos**`**
 - Um conjunto disjunto (``cdElemento``) é criado para cada vértice do grafo.
3. **`**Ordenação das Arestas**`**
 - Todas as arestas do grafo são ordenadas pelo peso, utilizando a função ``ordenar``.

4. ****Processamento das Arestas Ordenadas****

Para cada aresta `(u, v)` na lista ordenada:

- ****Verificação de Conjuntos****: Checa se `u` e `v` pertencem ao mesmo conjunto com `cdMesmoConjunto`.
- ****Adição à Árvore Geradora Mínima****: Se `u` e `v` não pertencem ao mesmo conjunto, a aresta `(u, v)` é adicionada à lista `a`, e os conjuntos que contêm `u` e `v` são unidos usando `cdUniao`.
- ****Atualização dos Representantes****: Após a união, o representante de cada vértice é atualizado.

5. ****Retorno da Árvore Geradora Mínima****

- Ao final, a lista `a` contém as arestas da árvore geradora mínima e é retornada.

Função `exibir(grafo: GrafoDirigido, arestas: list) -> None`

Esta função exibe o resultado da árvore geradora mínima:

- ****Primeira Linha****: Mostra o somatório dos pesos das arestas na árvore geradora mínima.
- ****Segunda Linha****: Exibe cada aresta da árvore no formato `"u-v"`, separadas por vírgulas.

O somatório é calculado somando os pesos das arestas presentes na lista `arestas`.